

Experiment No. 3

Title: Design Patterns in Python

Aim: Write a program to implement a configurable payment processing system using the strategy pattern.

Theory:

This program demonstrates a payment processing system where different payment methods (e.g., credit card, PayPal, Bitcoin) can be used interchangeably. The Strategy Pattern is employed to define a family of algorithms (payment methods), encapsulate each one, and make them interchangeable. This pattern allows the payment method to be selected at runtime.

- **Strategy Pattern:** This design pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it.
- **Payment Processing System:** A system that handles transactions from various payment methods, ensuring flexibility and scalability.

Experiment 3 Code:

```
class PaymentStrategy:
    def pay(self, amount):
        pass

class CreditCardPayment(PaymentStrategy):
    def pay(self, amount):
        print("Payment of ₹", amount, "processed using Credit Card.")

class DebitCardPayment(PaymentStrategy):
    def pay(self, amount):
        print("Payment of ₹", amount, "processed using Debit Card.")

class UPIPayment(PaymentStrategy):
    def pay(self, amount):
        print("Payment of ₹", amount, "processed using UPI.")

class PayPalPayment(PaymentStrategy):
    def pay(self, amount):
        print("Payment of ₹", amount, "processed using PayPal.")

class PaymentProcessor:
    def __init__(self, strategy):
```

```

        self.strategy = strategy

    def set_strategy(self, strategy):
        self.strategy = strategy

    def process_payment(self, amount):
        self.strategy.pay(amount)

print("_____")
print("    CONFIGURABLE PAYMENT PROCESSING")
print("_____")

amount = float(input("Enter payment amount: ₹"))

print("\nSelect Payment Method:")
print("1. Credit Card")
print("2. Debit Card")
print("3. UPI")
print("4. PayPal")

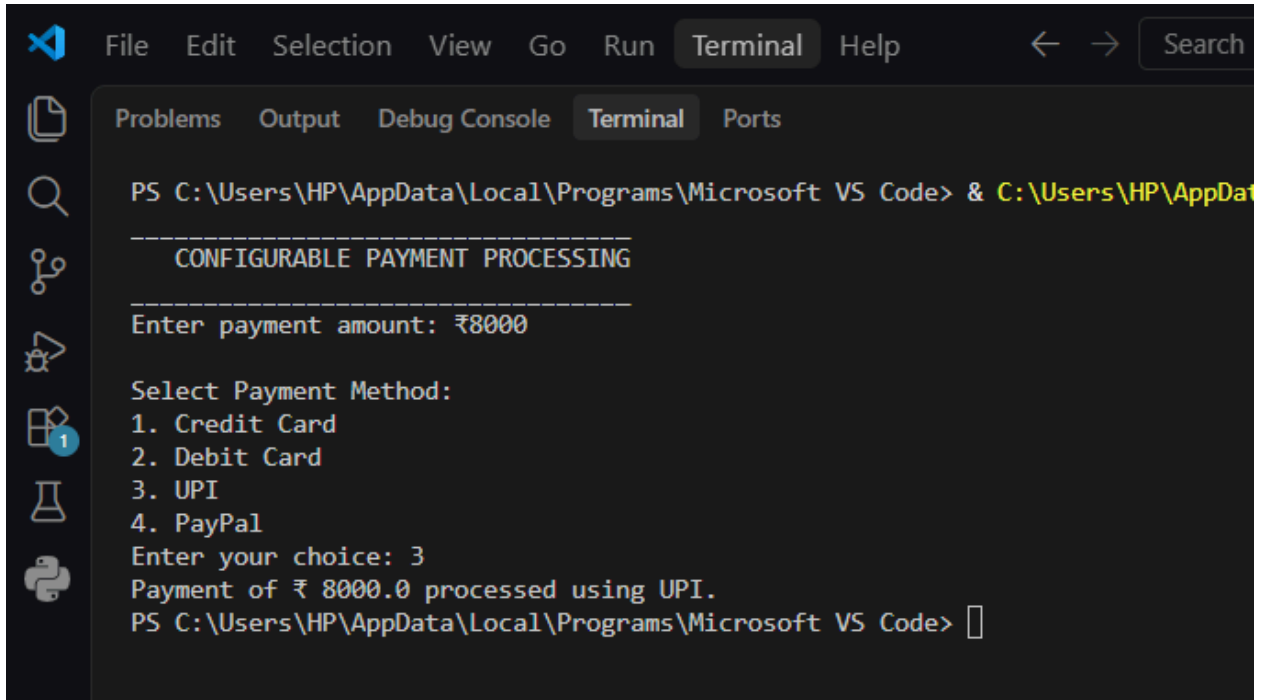
choice = int(input("Enter your choice: "))

if choice == 1:
    payment_method = CreditCardPayment()
elif choice == 2:
    payment_method = DebitCardPayment()
elif choice == 3:
    payment_method = UPIPayment()
elif choice == 4:
    payment_method = PayPalPayment()
else:
    print("Invalid payment method.")
    exit()

processor = PaymentProcessor(payment_method)
processor.process_payment(amount)

```

Experiment 3 Output (Screenshot)



```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
-----
CONFIGURABLE PAYMENT PROCESSING
-----
Enter payment amount: ₹8000

Select Payment Method:
1. Credit Card
2. Debit Card
3. UPI
4. PayPal
Enter your choice: 3
Payment of ₹ 8000.0 processed using UPI.
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> 
```

Conclusion:

The Strategy Pattern provides a flexible and scalable way to manage different payment methods in a payment processing system. By encapsulating each payment method within its own class, the system can easily switch between different algorithms at runtime without modifying the client code. This design pattern promotes the Open/Closed Principle, making the system open for extension but closed for modification.